Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 "Computer Science"

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»
Field of Study:
"Data Science and Artificial Intelligence"

Тема	Робастная миграция Java-сервисов на виртуальные потоки ИИ-агентами с детерминированными трансформациями кода
Topic	Robust Migration of Java Services to Virtual Threads by AI Agents with Deterministic Code Transformations

Работу выполнил / Prepared by	Трифонов М. К. M. K. Trifonov	_____ подпись / signature
Руководитель выпускной квалификационной работы / Final Qualification Work Supervisor	Садовых А. А. A. Sadovykh	_____ подпись / signature

Иннополис, Innopolis, 2027

Contents

1 Introduction **4**

1.1 Context and Relevance 4

1.2 Problem 5

1.3 Goal and Objectives 6

1.4 Object and Subject of the Research 7

1.5 Research Questions 7

1.6 Hypotheses 8

1.7 Novelty and Contributions 9

1.8 Scope 10

1.9 Thesis Outline 10

2 Literature Review **11**

2.1 Virtual Threads in the Java Platform 12

2.2 Hazards of Adopting Virtual Threads 13

2.2.1 Robustness of a Migration 13

2.2.2 Hazard Kinds 14

2.3 Code Migration and Its Automation 16

2.4 Model-Driven Modernization and Model Transformations 17

2.5 Deterministic Program Transformation 19

2.6 Language Models Combined with Deterministic Transformations . 20
2.7 Benchmarks for LLM-Based Coding Agents 21
2.8 Reliability of Generated Code 22
2.9 Failures Under Load and the Test Oracle 22
2.10 Feedback over Repeated Attempts 23
2.11 Experimental Methodology 23
2.12 Summary and Research Gap 24

Bibliography cited **25**

List of Tables

I Hazard Kinds Considered in This Thesis 15
II Benchmarks for the Migration of Java Code 22

Chapter 1

Introduction

1.1 Context and Relevance

Virtual threads became a final feature of the Java platform in JDK 21 [1]. They let a server keep the simple thread-per-request style without one operating-system thread per request: the JDK schedules virtual threads onto a small set of carrier threads and unmounts a virtual thread when it blocks. Spring Boot enables them with a single property, after which properties that configure thread pools have no effect [2]. Adopting virtual threads therefore looks like a configuration change, and it is often presented as one of the benefits of moving to Java 21.

AI coding agents now perform such migrations. Amazon Q Developer upgrades Java applications to Java 21 and lists virtual threads among the benefits [3], Google reports that LLM-based tools shortened its internal code migrations [4], [5], and four benchmarks published in 2025 and 2026 evaluate agents on the migration of Java code [6], [7], [8], [9]. The question whether these migrations keep the service working under load is therefore practical and current.

A migration to virtual threads is not only a version bump. The JDK documen-

tation states that virtual threads should not be pooled, that limiting concurrency is a job for semaphores, and that caching costly objects in thread-local variables can degrade performance because the object is then created for every virtual thread [1]. Blocking inside `synchronized` code holds the carrier thread on JDK 21 to 23 [10]. Mistakes of this kind have stopped production services [11], [12]. They share a property that matters for automated migration: the service still compiles and passes its tests.

1.2 Problem

Existing migration benchmarks judge an agent by whether the project builds and its tests pass [6], [7], [9], or by similarity to a reference migration [8]; none of them observes the migrated service under load. Test suites are known to overestimate the correctness of generated code [13], and concurrency problems that only degrade a service under load are a classic case of failures that functional checks do not observe [14], [15]. A migration to virtual threads can thus be counted as successful while it pins carrier threads, removes a limit that protected a downstream service, or never enables virtual threads. A preliminary study for this thesis observed exactly this: when coding agents edited the code themselves without guidance on hazards, practically every migration built and passed its tests, yet most were not robust (Chapter 5).

The usual remedies give the agent more instructions or a better check, and the agent still writes the code. Its result then differs from run to run, and a cheap model that lacks knowledge of the platform fails more often. Deterministic transformation tools, by contrast, give the same result every time, but their rules cannot decide questions of intent, such as whether a thread pool exists to protect a downstream

service. The existing deterministic recipe for this migration only enables virtual threads [16]. Recent work combines models with deterministic engines in three ways: the model writes the transformation rules, chooses among transformations that an engine applies, or edits the code while deterministic tools validate it [5], [17], [18], [19]. For this migration no tool lets a model take only the decisions that need judgement while deterministic transformations make every change, and no study compares these divisions of work in robustness, cost and the model size they need.

1.3 Goal and Objectives

The goal of this thesis is to make migrations of Java services to virtual threads by language models robust, by letting the model decide over a compact model of the service while deterministic code transformations make all changes, and to measure the robustness, cost and model-size boundary of this approach against the other divisions of work between a model and deterministic tools and against tools without a model. To reach the goal, the thesis pursues the following objectives:

1. define the robustness of a migration through resources observed at run time, and collect and classify the hazards of adopting virtual threads from the platform documentation, incident reports and the literature;

2. build a benchmark of runnable Spring Boot services with labelled hazards and reference migrations, and a dataset of real code from an open dataset of Java projects;

3. implement a robustness check that combines static rules with a short load run and runtime signals, and validate it on variants with known answers;

4. implement a migration tool that extracts a compact concurrency model, accepts a plan of decisions from a model or from rules, and applies deterministic transformations;

5. implement the other approaches of the literature for this migration: generic rules in a rewrite engine, a model that writes rewrite rules, a model that chooses transformations from the source code, and a model that edits the code;

6. run all approaches with models of different sizes from one family, repeat every model-based configuration in independent runs, with one attempt and with further attempts that receive feedback on the result, and evaluate robustness and cost.

1.4 Object and Subject of the Research

The *object* of the research is the migration of Java services to virtual threads performed by language models. The *subject* is the robustness of such migrations and the division of work between a language model and deterministic code transformations that makes them robust.

1.5 Research Questions

RQ1 Which division of work between a model and deterministic tools makes migrations robust: the model writes rewrite rules, chooses among deterministic transformations (from the source code or from a compact concurrency model), or edits the code itself, compared with tools that use no model; and how much do further attempts with feedback on the result help?

RQ2 From which model size on, within one model family, does each division of work produce robust migrations, and what does it cost in tokens and time?

RQ3 How well do the approaches apply to real code from an open dataset: which share of the relevant constructs does the migration tool find and transform, and how often do the actions taken by each approach agree with labelled decisions?

1.6 Hypotheses

The research questions lead to three hypotheses:

H1 When the model only chooses among deterministic transformations, the share of robust migrations is higher than when the same models write rewrite rules or edit the code, and higher than with any approach without a model except rules written for this migration. Further attempts with feedback raise the share of robust migrations for every division of work but do not remove the differences between them.

H2 Choosing among deterministic transformations works with smaller models than writing rules or editing the code, and needs fewer tokens and less time.

H3 On real code, most relevant constructs are found and can be transformed, the remaining errors of the tool come from decisions of intent rather than from the transformations, and approaches in which the model chooses agree with the labels more often than approaches in which it writes rules or edits the code.

In H1 the independent variables are the approach and the number of attempts and the dependent variable is the share of robust migrations; in H2 they are the

approach and the model size, and the share of robust migrations, the tokens and the time; in H3 the construct, and the shares of found and transformed constructs and of agreeing decisions.

1.7 Novelty and Contributions

To the best of our knowledge, based on the literature search described in Chapter 2, this is the first study of virtual-thread migrations performed by language models, the first tool that limits a model to decisions over a compact concurrency model for this migration, and the first comparison, on one task with one evaluation, of the three divisions of work between a model and deterministic tools described in the literature. The thesis makes four contributions:

- vtmigrate, a migration tool that extracts a compact concurrency model of a Spring Boot service, takes one decision per element from a closed vocabulary, and applies deterministic transformations of the syntax tree and generated helper classes; it works with any model and, with built-in rules, without one;

- vtcheck, a robustness check that combines static rules with a load run based on JDK Flight Recorder events, thread dumps and a counting stub of the downstream service, usable as a benchmark oracle, as a tool for the agent and as a step in continuous integration;

- a benchmark of runnable migration tasks with labelled hazards and reference migrations, and a dataset of real code taken from an open dataset of Java projects;

- an evaluation that compares tools without a model, a model that writes rewrite rules, a model that chooses transformations and a model that edits the code, across a ladder of model sizes from one family, with one attempt and with feedback over further attempts, over independent runs, in robustness, tokens and time.

1.8 Scope

In scope are Spring Boot services on JDK 21, migrations to virtual threads, their robustness under concurrent load, and one family of models that run on a laptop. Out of scope are building a general-purpose agent, security in the sense of protection against attacks, reactive stacks, JDK versions from 24 on, where `synchronized` no longer pins [10], and the question of what size a concurrency limit should have; the thesis keeps the limits that a service had before the migration.

1.9 Thesis Outline

Chapter 2 reviews virtual threads and their hazards, defines robustness, and reviews automated migration, model-driven modernization, deterministic program transformation, their combination with language models, and benchmarks for coding agents. Chapter 3 describes the benchmark, the dataset, the approaches compared and the evaluation design. Chapter 4 presents the benchmark generator, the robustness check, the migration tool and the harness. Chapter 5 validates the check and answers the research questions, and Chapter 6 concludes.

Chapter 2

Literature Review

This chapter reviews the work this thesis builds on. Section 2.1 describes virtual threads and what is known about their behaviour in server frameworks. Section 2.2 defines the robustness of a migration and collects the hazards of adopting virtual threads from the platform documentation and incident reports. Section 2.3 turns to automated migration, Section 2.4 to model-driven modernization and model transformations, Section 2.5 to deterministic program transformation tools, and Section 2.6 to approaches that combine language models with such tools. Section 2.7 covers benchmarks for LLM-based coding agents, including the recent benchmarks for Java migration, and Section 2.8 the reliability of generated code. Section 2.9 covers failures under load and the test oracle, Section 2.10 feedback over repeated attempts, and Section 2.11 experimental methodology. Section 2.12 states the gap this thesis addresses.

2.1 Virtual Threads in the Java Platform

Virtual threads became a final feature of the Java platform in JDK 21 [1]. A virtual thread is an instance of java.lang.Thread that is not tied to an operating-system thread for its whole lifetime. The JDK runs virtual threads on a small pool of platform threads, called carrier threads; when a virtual thread blocks on I/O or on a java.util.concurrent lock, it is unmounted from its carrier, which can then run another virtual thread [1]. The goal stated in JEP 444 is to let server applications written in the simple thread-per-request style scale with near-optimal hardware utilisation, without moving to asynchronous or reactive programming [1].

The same JEP lists the conditions under which a virtual thread cannot be unmounted and therefore holds its carrier, a situation called *pinning*: running inside a synchronized block or method, and running a native method or a foreign function [1]. In JDK 21 to 23 a virtual thread that blocks inside synchronized code keeps the carrier busy for the whole time it waits. JEP 491 removed this limitation for synchronized in JDK 24; pinning remains possible during class initialisation and in calls from native code [10]. Long-term-support releases matter here: a service that runs on JDK 21 is exposed to the original behaviour.

Frameworks integrate virtual threads in different ways. In Spring Boot they are enabled with the property spring.threads.virtual.enabled; the documentation states that properties which configure thread pools then have no effect, strongly recommends Java 24 or later, points to pinned virtual threads as a cause of lower throughput, and warns that virtual threads are daemon threads, so an application kept alive only by scheduled tasks may exit unless spring.main.keep-alive is set [2]. That the embedded Tomcat server handles requests on virtual threads after this change is confirmed by the thread dumps of the validation in Chapter 5. Navarro

et al. describe the integration of virtual threads into the Quarkus framework and benchmark it in a container with scarce resources; in their study the variant based on virtual threads did not perform as well as the reactive variant, which they attribute to a mismatch between the assumptions of Netty and of virtual threads about the number of threads [20]. A further study compares virtual and platform threads in database-driven server applications and reports preliminary results [21].

In our interpretation, what distinguishes virtual threads from other forms of lightweight concurrency is that existing blocking code keeps its form, and this is also why adopting them is easy to underestimate: the code compiles and looks the same, while its behaviour under load can change.

The academic literature on virtual threads is small. The papers found for this review are performance comparisons and framework integration reports, such as [20], [21]; none of them studies how existing services are migrated or which mistakes are made in such migrations.

2.2 Hazards of Adopting Virtual Threads

2.2.1 Robustness of a Migration

Words such as *safe* are read differently by different communities. In the taxonomy of dependable and secure computing, safety is the “absence of catastrophic consequences on the user(s) and the environment”, one attribute of dependability next to availability, reliability, integrity and maintainability, and robustness is a secondary attribute: “dependability with respect to external faults” [22]. The systems and software engineering vocabulary defines robustness as “the degree to which a system or component can function correctly in the presence of invalid inputs or stressful environmental conditions” [23]. This thesis is not concerned with

security or with catastrophic consequences, but with whether a migrated service keeps working, and keeps its neighbours working, under concurrent load, which is a stressful environmental condition in the sense of the vocabulary. It therefore uses the term *robustness* and defines it through resources observed at run time.

Definition 1 (Robust migration)**.** A migration of a service to virtual threads is *robust* if, under a specified concurrent load, the migrated service (1) handles requests on virtual threads; (2) never holds a carrier thread while a virtual thread is blocked in application code; (3) never uses any external resource, such as a downstream service, a database or a pool, with more concurrent requests than the service before the migration allowed; (4) stays alive and answers requests with an error rate of at most 1%; and (5) does not depend on a library version with a known problem under virtual threads.

Conditions (1) to (4) are observable at run time; condition (5) is checked statically because its effect needs production infrastructure to appear. The definition is the thesis's own operationalisation and is used by the robustness check of Chapter 4. A migration *method*, such as an agent with a given model or a migration tool, is the more robust the larger the share of its independent runs that yield a robust migration; a deterministic method produces the same result in every run.

2.2.2 Hazard Kinds

This thesis uses the term *hazard* for a change, or a missing change, after which a service still compiles and passes its tests but violates Definition 1 or degrades under load. Table I lists the hazard kinds considered in this thesis and the sources that describe them.

Pinning. Netflix reported services that stopped serving requests after moving

TABLE I
Hazard Kinds Considered in This Thesis

Kind	Description	Sources
PIN	Blocking while holding a monitor pins the carrier thread on JDK 21–23.	[1], [10], [11]
LIMIT	A thread pool that also limited concurrent calls to a resource is replaced by an unbounded executor.	[1]
POOL	Virtual threads are placed in a fixed-size pool.	[1]
TLOCAL	A ThreadLocal cache of a costly object is rebuilt for every virtual thread.	[1]
CONF	Virtual threads are not enabled, or configured with a property that does not exist or has no effect.	[2]
DEP	A dependency version has a known problem under virtual threads.	[12], [24]
DAEMON	A scheduler-only application exits because virtual threads are daemon threads.	[2]

to JDK 21 and virtual threads; the investigation showed carrier threads held by virtual threads that were waiting for a lock inside synchronized code [11]. The problem is invisible in functional tests because it needs enough concurrent requests to occupy all carriers.

Removed limits and pooling. JEP 444 states that virtual threads should not be pooled and that a semaphore is the construct for limiting concurrency [1]. In code written for platform threads a fixed-size pool often has two roles: it reuses expensive threads and it limits how many tasks access a resource at the same time. A migration that follows the advice “do not pool virtual threads” literally removes the second role together with the first. A migration that keeps the pool but fills it with virtual threads keeps the limit but contradicts the platform guidance.

Thread-local caches. Per-thread caches of costly objects, such as date formats or message digests, assume a small set of long-lived threads. With one virtual thread per task they are created for every task, which can degrade performance significantly [1]. It costs memory and time but does not violate Definition 1, and

the thesis treats it as low severity.

Configuration. A virtual-thread migration adds two new ways to get configuration wrong: setting a property that does not exist, after which the service runs exactly as before, and keeping thread-pool properties that silently stop having an effect [2]. When such a property was the only limit on concurrent requests, its loss is a removed limit hidden in configuration.

Dependencies. Libraries change under virtual threads too. HikariCP 7.0.2 was reported to saturate all carrier threads under virtual-thread load, and the problem was fixed in version 7.1.0 [12]; the PostgreSQL JDBC driver 42.7.8 was reported to pin virtual threads [24]. Such problems can be reported after the training cut-off of a model, which then cannot know about them (interpretation).

Daemon threads. Virtual threads are daemon threads; Spring Boot therefore offers a `keep-alive` property for applications without a web server [2]. A scheduler-only service that only enables virtual threads exits right after start-up.

The hazards differ in when they appear. Some are *latent*: they are already in the code before the migration and become harmful once virtual threads are enabled (pinning, thread-local caches, dependency versions, scheduler-only applications, limits that live only in server configuration). Others are *traps*: the code is harmless as long as the migration does not change it, for example a bounded pool that the agent may replace with an unbounded executor. Both kinds are used in the benchmark of Chapter 3.

2.3 Code Migration and Its Automation

Migrating code to a new API or platform version is a long-standing software engineering problem. Concurrencer refactors sequential Java code to use

java.util.concurrent; its authors observe that such a manual migration is tedious, error-prone and omission-prone, and that a find-and-replace tool cannot perform it [25]. A closely related migration is the move from synchronous to asynchronous APIs in JavaScript. Desynchronizer relies on a static analysis that may be unsound, so its refactorings are presented as suggestions that the programmer must review [26]. Both tools show a recurring theme: migrations that change concurrency need analysis beyond syntax, and their results need checking.

Large language models are now used for migrations in industry. Google reports in an experience report that LLM-based migration reduced the time needed for migrations significantly; the authors stress that the report is not a controlled study [4]. Amazon Q Developer offers automated upgrades of Java applications to Java 21 and presents virtual threads among the benefits of the new version [3].

2.4 Model-Driven Modernization and Model Transformations

Migrating a system by first building a model of it is the approach of model-driven modernization. The Object Management Group (OMG) calls it Architecture-Driven Modernization (ADM) and has standardised a metamodel for it: the Knowledge Discovery Metamodel (KDM), adopted as ISO/IEC 19506, "defines a metamodel for representing existing software assets, their associations, and operational environments" [27]. MoDisco, the Eclipse framework for model-driven reverse engineering, implements several ADM standards, including KDM. It discovers an initial model from the legacy artefacts with technology-specific *model discoverers* and then derives further models and views from it [28]. Its Java discoverer, however, has not followed the language: the review of release 1.6.0 (March 2026)

states that there is “no support for Java evolution since Java 5”, that the prediction that discovery of modern Java would fail was confirmed, and that the project has moved to life support and considers a termination review [29]. Services on Java 21 therefore cannot be discovered with it. The REMICS project applied the same idea to migrating legacy applications to service clouds: its process consists of understanding the legacy system, designing the new application and verifying and implementing it, with tool support for recovery and forward engineering [30].

The engine of model-driven engineering is the model transformation. Model transformations are classified as model-to-model or model-to-text, and Czarnecki and Helsen give a feature model of the design choices of transformation approaches [31]. ATL is a widely used model-to-model transformation language and tool [32]; MOFScript was one of the submissions to OMG’s standard for model-to-text transformations, assessed against requirements for such languages [33]. Two further techniques relate models to each other rather than transforming them. *Model weaving* captures links between elements of different models in a separate weaving model, from which transformations can be derived [34]. *Ontologies* make the concepts behind a metamodel explicit: Kappel et al. lift metamodels into ontologies so that modelling languages can be matched on a conceptual level and the mappings can serve as a basis for transformation code [35].

The migration tool of this thesis is built on these ideas (interpretation). Its extraction step is a small model discoverer that produces a model of the concurrency constructs of a service; the decisions of a model or of rules form a weaving model that links each element to one action of a closed vocabulary; and the actions are model-to-model rewrites of the syntax tree and model-to-text templates for generated classes. The catalogue of hazards plays the role of a small ontology that relates code constructs to the run-time resources they consume. Because MoDisco

cannot discover Java 21 code, the model-driven approach enters the comparison of this thesis through this lightweight pipeline rather than through MoDisco and ATL.

2.5 Deterministic Program Transformation

Source-to-source transformation tools change code by rules that are written once and then applied repeatedly with the same result. Refaster lets developers specify a Java refactoring by compilable before-and-after examples and has been used across Google's Java code base [36]. Comby matches and rewrites syntactic fragments in many languages by declarative templates [37]. OpenRewrite applies recipes to a lossless syntax tree of Java projects and offers a recipe that enables virtual threads in Spring Boot: it sets `spring.threads.virtual.enabled=true` for projects on Java 21 or later and does nothing else [16]. The other OpenRewrite recipes for virtual threads only search: they mark implementations and constructors of `Thread` and the creation of platform-thread executors, and do not change them [38]. MigrationBench uses an OpenRewrite recipe for the upgrade to Java 17 as a static baseline without generative AI; it costs no model calls but achieves a low success rate on maximal migration [6].

The common property of these tools is determinism: a rule gives the same output for the same input, can be reviewed once and reused. Their limitation is that someone must write the rule, and a rule cannot decide questions that depend on the intent of the code, for example whether a thread pool exists to reuse threads or to protect a downstream service.

2.6 Language Models Combined with Deterministic Transformations

Several recent works combine a language model with a deterministic transformation engine and divide the work between them differently. In one group the model writes the rules. MELT infers Comby rules for API migrations from pull requests of libraries and uses a model to generate additional code examples [17]. Allain et al. let three models synthesise rules in Comby, GritQL and Ast-Grep and argue that rules mitigate the limited explainability, the lack of determinism and the cost of models on repetitive tasks; the largest model produced the most applicable and correct rules, while the smaller ones were effective for simple, localised changes and struggled with complex migrations [18]. BigBag lets a coding agent write a Spoon or JavaParser transformation that fixes a breaking dependency update in one client and then applies it to other clients; the choice of the engine affected how often the generated transformation compiled [39]. In a second group the model proposes and an established engine executes. EM-Assist asks a model for Extract Method suggestions, filters invalid and useless ones, and applies the selected one with the refactoring engine of IntelliJ IDEA [19]. In a third group the model edits the code and deterministic tools find the places and validate the result: Google's migration of identifiers from 32 to 64 bits combines change location discovery, a model that produces the edits and several forms of validation [5].

These works suggest a design space: the model can write the transformation, choose among transformations, or perform the edit itself. A position paper from NVIDIA argues that small language models are sufficient and more economical for the narrow, repetitive invocations that dominate agentic systems [40], which favours designs in which the model's task is small. None of the works above targets

the concurrency model of a program, and none checks the result under load. The experiment of this thesis instantiates each of the three groups for the migration to virtual threads and compares them, on the same tasks and with the same evaluation, to each other and to tools without a model (Chapter 3).

2.7 Benchmarks for LLM-Based Coding Agents

Function-level benchmarks such as HumanEval check generated code by tests. EvalPlus showed that such tests can be insufficient: extending the HumanEval benchmark with 80 times more tests reduced pass@k by up to 28.9% and changed the ranking of models [13]. This result is central for this thesis: the strength of the oracle determines what a benchmark measures.

Four recent benchmarks evaluate the migration of Java code (Table II). MigrationBench contains 5,102 repositories, with a selected subset of 300, for migration from Java 8 to Java 17 or 21; on the selected subset an agentic framework with Claude-4.5-Sonnet reached a success rate of 71.67% for minimal and 53.33% for maximal migration [6]. FreshBrew evaluates agents on project-level migrations of 228 repositories to JDK 17 and argues that avoiding reward hacking requires projects with high test coverage; the best model migrated 52.3% of the projects [7]. JMigBench evaluates function-level migration from Java 8 to Java 11 with similarity metrics [8]. ScarfBench evaluates migration between Spring, Jakarta EE and Quarkus with an executable oracle that compiles, deploys and runs behavioural tests; the strongest agent passed 15.3% of the tests on single-layer tasks [9].

None of these benchmarks targets the concurrency model. Three of them check that the program builds and behaves the same in tests, and JMigBench compares the result with a reference migration; for a version upgrade these are the

TABLE II
Benchmarks for the Migration of Java Code

Benchmark	Migration	Size	Success criterion
MigrationBench [6]	Java 8 to 17 or 21	5,102 repositories	build and tests
FreshBrew [7]	to JDK 17	228 repositories	build and tests, high coverage
JMigBench [8]	Java 8 to 11, functions	function pairs	similarity metrics
ScarfBench [9]	between frameworks	204 tasks	build, deployment, behavioural tests
This thesis	to virtual threads	10 services, 48 modules	build, tests, robustness check, load run

right questions. For a change of the concurrency model it is not enough, because the behaviour that changes is the behaviour under concurrent load.

2.8 Reliability of Generated Code

Generated code can name things that do not exist: across 16 models and 576,000 samples the average share of hallucinated packages was at least 5.2% for commercial and 21.7% for open-source models [41]. A non-existent configuration property is the same failure in another place.

For concurrent code, CONCUR provides 115 problems, 43 derived from a concurrency textbook and 72 validated mutants, and its evaluation highlights limitations of current models [42]. None of these studies covers virtual threads or migrations of existing services.

2.9 Failures Under Load and the Test Oracle

Many hazards of this thesis show up only as degraded service, not as crashes. Huang et al. call such failures gray failures and identify differential observability

as their key feature: the system's failure detectors may not notice a problem that affects applications [14]. Detecting them requires an oracle that observes the right signal. The test oracle problem, deciding whether observed behaviour is correct, is a known bottleneck of test automation [15]. The robustness check in this thesis is a small load test whose oracle is defined by runtime signals rather than by functional assertions.

2.10 Feedback over Repeated Attempts

Language agents can improve from feedback without retraining. Reflexion agents reflect on task feedback in natural language and keep the reflections in memory [43]; Self-Refine lets a model critique and refine its own output and improves results by about 20% on average across the studied tasks [44]. The experiment of this thesis therefore gives each division of work further attempts with feedback from the robustness check. Whether a model acts on the findings, or works around them, is an empirical question addressed by RQ1.

2.11 Experimental Methodology

Le Boudec gives a general treatment of performance evaluation and the presentation of results with confidence intervals [45]. Agents add non-determinism of their own. Guidelines for empirical studies in software engineering that involve LLMs recommend reporting exact model versions, configurations and the dates of execution, not treating a fixed temperature as a guarantee of reproducibility, and including an open-weight model so that at least part of a study can be replicated [46]. Recent migration benchmarks report a single run per task [7], [9]. The

evaluation in this thesis follows these recommendations, includes an open-weight model family and repeats runs to report intervals.

2.12 Summary and Research Gap

The review supports five observations. First, the platform documentation and incident reports describe specific hazards of adopting virtual threads, and the academic literature on virtual threads does not study migrations [1], [10], [11], [20]. Second, AI coding agents are already evaluated on Java migrations, but the benchmarks judge success by building and testing the project or by similarity to a reference, which does not observe behaviour under load [6], [7], [8], [9]. Third, generated code is known to have defects that functional checks miss, including invented names [13], [41], and concurrency problems that degrade a service need oracles based on runtime signals rather than assertions [14], [15]. Fourth, model-driven modernization and deterministic program transformation offer reproducible migrations, but their rules cannot decide questions of intent [16], [28], [37]. Fifth, recent work combines language models with deterministic engines, mostly by letting the model write the rules or propose refactorings, and reports that smaller models struggle when the task is complex [18], [19], [39].

No study measures how robust agent migrations to virtual threads are, and no tool lets a model take only the decisions of such a migration while deterministic transformations make every change. It is also unknown how small a model can be when its input and output are reduced to a compact model and a plan. This thesis addresses that gap with a migration tool built on deterministic transformations, a run-time robustness check, a benchmark and a dataset of real code, and an evaluation across model sizes.

Bibliography cited

[1] R. Pressler and A. Bateman. “JEP 444: Virtual Threads,” OpenJDK, Accessed: Oct. 4, 2026. [Online]. Available: https://openjdk.org/jeps/444

[2] Spring Boot Team. “Spring Boot Reference Documentation: SpringApplication,” Accessed: Oct. 4, 2026. [Online]. Available: https://docs.spring.io/spring-boot/reference/features/spring-application.html

[3] B. Beach. “Announcing support for upgrades to Java 21 in Amazon Q Developer,” AWS DevOps Blog, Accessed: Oct. 4, 2026. [Online]. Available: https://aws.amazon.com/blogs/devops/announcing-support-for-upgrades-to-java-21-in-amazon-q-developer/

[4] S. Nikolov et al., “How is Google Using AI for Internal Code Migrations?” In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, IEEE, Apr. 2025, pp. 481–492. DOI: 10.1109/icse-seip66354.2025.00048

[5] C. Ziftci, S. Nikolov, A. Sjövall, B. Kim, D. Codecasa, and M. Kim, “Migrating code at scale with llms at google,” in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion ’25, ACM, 2025, pp. 162–173. DOI: 10.1145/3696630.3728542 [Online]. Available: http://dx.doi.org/10.1145/3696630.3728542

[6] L. Liu et al., "MigrationBench: Repository-Level Code Migration Benchmark from Java 8," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, ACM, Aug. 2026, pp. 9427–9438. DOI: 10.1145/3770855.3817526

[7] V. May, D. Misra, Y. Luo, A. Sridhar, J. Gehring, and S. S. Ribeiro, "FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, ACM, Apr. 2026, pp. 944–955. DOI: 10.1145/3744916.3773214

[8] N. Amin, Z. Fei, X. Li, J. Petke, and H. Ye, *JMigBench: A Benchmark for Evaluating LLMs on Source Code Migration (Java 8 to Java 11)*, 2026. DOI: 10.48550/ARXIV.2602.09930 [Online]. Available: https://arxiv.org/abs/2602.09930

[9] A. Pavuluri et al., *ScarfBench: A Benchmark for Cross-Framework Application Migration in Enterprise Java*, 2026. DOI: 10.48550/ARXIV.2605.06754 [Online]. Available: https://arxiv.org/abs/2605.06754

[10] P. Chilano Mateo and A. Bateman. "JEP 491: Synchronize Virtual Threads without Pinning," OpenJDK, Accessed: Oct. 4, 2026. [Online]. Available: https://openjdk.org/jeps/491

[11] V. Filanovsky, M. Huang, D. Thomas, and M. Chalupa. "Java 21 Virtual Threads - Dude, Where's My Lock?" Netflix Technology Blog, Accessed: Oct. 4, 2026. [Online]. Available: https://netflixtechblog.com/java-21-virtual-threads-dude-wheres-my-lock-3052540e231d

[12] "HikariCP 7.0.2: ConcurrentBag.requite() yield-spin still saturates all carrier threads under virtual thread load," GitHub, HikariCP issue #2398,

Accessed: Oct. 4, 2026. [Online]. Available: https://github.com/brettwooldridge/HikariCP/issues/2398

[13] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in *Advances in Neural Information Processing Systems 36*, Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 21 558–21 572. DOI: 10.52202/075280-0943

[14] P. Huang et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems," in *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, ACM, May 2017, pp. 150–155. DOI: 10.1145/3102980.3103005

[15] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The Oracle Problem in Software Testing: A Survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, May 2015, ISSN: 1939-3520. DOI: 10.1109/tse.2014.2372785

[16] Moderne. "OpenRewrite recipe catalog: Enable Virtual Threads on Java 21 (org.openrewrite.java.spring.boot3.EnableVirtualThreads)," Accessed: Oct. 21, 2026. [Online]. Available: https://docs.openrewrite.org/recipes/java/spring/boot3/enablevirtualthreads

[17] D. Ramos, H. Mitchell, I. Lynce, V. Manquinho, R. Martins, and C. L. Goues, "Melt: Mining effective lightweight transformations from pull requests," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2023, pp. 1516–1528. DOI: 10.1109/ase56229.2023.00117 [Online]. Available: http://dx.doi.org/10.1109/ase56229.2023.00117

[18] A. Allain, A. Blot, D. E. Khelladi, and M. Acher, "Code transformation rule synthesis using llms: Potential and limits," in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '26, ACM, 2026, pp. 2639–2651. DOI: 10.1145/3832783.3837504 [Online]. Available: http://dx.doi.org/10.1145/3832783.3837504

[19] D. Pomian et al., "Em-assist: Safe automated extractmethod refactoring with llms," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE '24, ACM, 2024, pp. 582–586. DOI: 10.1145/3663529.3663803 [Online]. Available: http://dx.doi.org/10.1145/3663529.3663803

[20] A. Navarro, J. Ponge, F. Le Mouël, and C. Escoffier, "Considerations for integrating virtual threads in a Java framework: a Quarkus example in a resource-constrained environment," in *Proceedings of the 17th ACM International Conference on Distributed and Event-based Systems*, ACM, Jun. 2023, pp. 103–114. DOI: 10.1145/3583678.3596895

[21] L. Lasić, D. Beronić, B. Mihaljević, and A. Radovan, "Assessing the Efficiency of Java Virtual Threads in Database-Driven Server Applications," in *2024 47th MIPRO ICT and Electronics Convention (MIPRO)*, IEEE, May 2024, pp. 2045–2050. DOI: 10.1109/mipro60963.2024.10569754

[22] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004, ISSN: 1545-5971. DOI: 10.1109/tdsc.2004.2 [Online]. Available: http://dx.doi.org/10.1109/tdsc.2004.2

[23] *ISO/IEC/IEEE 24765:2010 Systems and software engineering – Vocabulary*, Definition of robustness as quoted in the SEBoK glossary, https://sebokwiki.org/wiki/Robustness_(glossary), ISO/IEC/IEEE, 2010. Accessed: Oct. 21, 2026.

[24] "Virtual Thread pinned in PostgreSQL JDBC Driver 42.7.8," GitHub, pgjdbc issue #3835, Accessed: Oct. 4, 2026. [Online]. Available: https://github.com/pgjdbc/pgjdbc/issues/3835

[25] D. Dig, J. Marrero, and M. D. Ernst, "Refactoring sequential Java code for concurrency via concurrent libraries," in *2009 IEEE 31st International Conference on Software Engineering*, IEEE, 2009, pp. 397–407. DOI: 10.1109/icse.2009.5070539

[26] S. Gokhale, A. Turcotte, and F. Tip, "Automatic migration from synchronous to asynchronous JavaScript APIs," *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–27, Oct. 2021, ISSN: 2475-1421. DOI: 10.1145/3485537

[27] *ISO/IEC 19506:2012 Information technology – Object Management Group Architecture-Driven Modernization (ADM) – Knowledge Discovery Meta-Model (KDM)*, ISO/IEC, 2012. Accessed: Oct. 21, 2026. [Online]. Available: https://www.iso.org/standard/32625.html

[28] H. Brunelière, J. Cabot, G. Dupé, and F. Madiot, "Modisco: A model driven reverse engineering framework," *Information and Software Technology*, vol. 56, no. 8, pp. 1012–1032, 2014, ISSN: 0950-5849. DOI: 10.1016/j.infsof.2014.04.007 [Online]. Available: http://dx.doi.org/10.1016/j.infsof.2014.04.007

[29] Eclipse Foundation. "Eclipse MoDisco 2026-03 (1.6.0) Release Review," Accessed: Oct. 24, 2026. [Online]. Available: https://projects.eclipse.org/projects/modeling.modisco/releases/2026-03-1.6.0/review

[30] P. Mohagheghi, A. J. Berre, A. Henry, F. Barbier, and A. Sadovykh, "Remics-reuse and migration of legacy applications to interoperable cloud services," in *Towards a Service-Based Internet*. Springer Berlin Heidelberg, 2010, pp. 195–196, ISBN: 9783642176944. DOI: 10.1007/978-3-642-17694-4_20 [Online]. Available: http://dx.doi.org/10.1007/978-3-642-17694-4_20

[31] K. Czarnecki and S. Helsen, "Feature-based survey of model transformation approaches," *IBM Systems Journal*, vol. 45, no. 3, pp. 621–645, 2006, ISSN: 0018-8670. DOI: 10.1147/sj.453.0621 [Online]. Available: http://dx.doi.org/10.1147/sj.453.0621

[32] F. Jouault, F. Allilaire, J. Bézivin, and I. Kurtev, "Atl: A model transformation tool," *Science of Computer Programming*, vol. 72, no. 1-2, pp. 31–39, 2008, ISSN: 0167-6423. DOI: 10.1016/j.scico.2007.08.002 [Online]. Available: http://dx.doi.org/10.1016/j.scico.2007.08.002

[33] J. Oldevik, T. Neple, R. Grønmo, J. Aagedal, and A.-J. Berre, "Toward standardised model to text transformations," in *Model Driven Architecture – Foundations and Applications*. Springer Berlin Heidelberg, 2005, pp. 239–253, ISBN: 9783540320937. DOI: 10.1007/11581741_18 [Online]. Available: http://dx.doi.org/10.1007/11581741_18

[34] M. Didonet Del Fabro and P. Valduriez, "Towards the efficient development of model transformations using model weaving and matching transformations," *Software & Systems Modeling*, vol. 8, no. 3, pp. 305–324, 2008,

ISSN: 1619-1374. DOI: 10.1007/s10270-008-0094-z [Online]. Available: http://dx.doi.org/10.1007/s10270-008-0094-z

[35] G. Kappel et al., "Lifting metamodels to ontologies: A step to the semantic integration of modeling languages," in *Model Driven Engineering Languages and Systems*. Springer Berlin Heidelberg, 2006, pp. 528–542, ISBN: 9783540457732. DOI: 10.1007/11880240_37 [Online]. Available: http://dx.doi.org/10.1007/11880240_37

[36] L. Wasserman, "Scalable, example-based refactorings with refaster," in *Proceedings of the 2013 ACM workshop on Workshop on refactoring tools*, ser. SPLASH '13, ACM, 2013, pp. 25–28. DOI: 10.1145/2541348.2541355 [Online]. Available: http://dx.doi.org/10.1145/2541348.2541355

[37] R. van Tonder and C. Le Goues, "Lightweight multi-language syntax transformation with parser parser combinators," in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '19, ACM, 2019, pp. 363–378. DOI: 10.1145/3314221.3314589 [Online]. Available: http://dx.doi.org/10.1145/3314221.3314589

[38] Moderne. "OpenRewrite recipe catalog: Find Virtual Thread opportunities (org.openrewrite.java.migrate.lang.FindVirtualThreadOpportunities)," Accessed: Oct. 24, 2026. [Online]. Available: https://docs.openrewrite.org/recipes/java/migrate/lang

[39] F. Reyes, B. Baudry, and M. Monperrus, *Agentic generation of ast transformation rules for fixing breaking updates*, 2026. DOI: 10.48550/ARXIV.2606.24446 [Online]. Available: https://arxiv.org/abs/2606.24446

[40] P. Belcak et al., *Small language models are the future of agentic ai*, 2025. DOI: 10.48550/ARXIV.2506.02153 [Online]. Available: https://arxiv.org/abs/2506.02153

[41] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala, "We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs," in *34th USENIX Security Symposium (USENIX Security 25)*, USENIX Association, 2025, pp. 3687–3706. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen

[42] J. Huang, T. Mahmud, C. S. Pasareanu, and G. Yang, "CONCUR: Benchmarking LLMs for Concurrent Code Generation," *Proceedings of the ACM on Software Engineering*, vol. 3, no. ISSTA, pp. 4474–4494, Oct. 2026, ISSN: 2994-970X. DOI: 10.1145/3832286

[43] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36*, Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 8634–8652. DOI: 10.52202/075280-0377

[44] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems 36*, Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 46 534–46 594. DOI: 10.52202/075280-2019

[45] J.-Y. Le Boudec, *Performance Evaluation of Computer and Communication Systems*, Version 2.3 of April 4, 2022. 2022. [Online]. Available: https://leboudec.github.io/perfeval/

[46] S. Baltes et al. “Guidelines for Empirical Studies in Software Engineering involving Large Language Models.” Accepted in Empirical Software Engineering; living version at https://llm-guidelines.org. arXiv: 2508.15503.